資料結構・課堂 3　Stack 堆疊　測驗

一、選擇題　（每題 5 分，共 25 分）

1. Stack 的核心特性是？

(　　) A. FIFO（先進先出）

(　　) B. LIFO（後進先出）

(　　) C. 隨機存取任意元素

(　　) D. 雙向操作（兩端均可進出）

---

2. 依序執行：`push(5)` → `push(10)` → `push(3)` → `pop()` → `top()`，回傳值為？

(　　) A. 3　B. 5　C. 10　D. 空

---

3. 使用 Array 實作 Stack，`push` 的時間複雜度是？

(　　) A. O(n)　B. O(log n)　C. O(1)　D. $O(n^2)$

---

4. 關於 C++ STL 的 `std::stack`，以下哪個說法**正確**？

(　　) A. `pop()` 會回傳被移除的元素值

(　　) B. 應先呼叫 `top()` 取值，再呼叫 `pop()` 移除

(　　) C. `push()` 的時間複雜度是 O(n)

(　　) D. Stack 可以用 index 直接存取任意元素

---

5. 下列哪個場景**最適合**使用 Stack？

(　　) A. CPU 行程排程（先來先執行）

(　　) B. 印表機列印佇列

(　　) C. 撤銷（Undo）操作記錄

(　　) D. 廣度優先搜尋（BFS）

二、填空題　（每格 4 分，共 24 分）

6. Stack 的五個 ADT 核心操作（全部皆為 O(1)）：

操作	說明
`push(x)`	將元素 x 放入 _ _ _ _ _ _
`pop()`	_ _ _ _ _ _ 並回傳頂端元素
`top()`	_ _ _ _ _ _ 頂端元素（不移除）
`isEmpty()`	檢查 Stack 是否為 _ _ _ _ _ _
`size()`	回傳元素 _ _ _ _ _ _

7. 遞迴呼叫太深會造成 **_ _ _ _ _ _ Overflow**，

原因是系統的 _ _ _ _ _ _ Stack 空間被耗盡。

8. 括號匹配演算法三步驟：

① 遇到**左**括號時，將其 _ _ _ _ _ _ 進 Stack

② 遇到**右**括號時，從 Stack _ _ _ _ _ _ 一個元素並檢查是否配對

③ 最後若 Stack 為 _ _ _ _ _ _，代表所有括號配對正確

三、操作追蹤題　（每題 12 分，共 24 分）

9. 追蹤以下操作，填入每個步驟後 Stack 的狀態（由底至頂）與輸出值。

操作序列：`push(10)` → `push(25)` → `push(7)` → `top()` → `pop()` → `push(99)` → `pop()` → `pop()`

步驟	操作	Stack 內容（底 → 頂）	輸出
1	push(10)		—
2	push(25)		—
3	push(7)		—
4	top()		

5	pop()		
6	push(99)		—
7	pop()		
8	pop()		

---

10. 利用 Stack 檢查 `"{[()]}"` 的括號配對，填寫每個步驟的狀態。

字元	動作	Stack 狀態（底 → 頂）	配對正確？
`{`	push	`{ }`	—
`[`	push		—
`(`	push		—
`)`	pop 出 _ _，與 `)` 比對		☐ 是　☐ 否
`]`	pop 出 _ _，與 `]` 比對		☐ 是　☐ 否
`}`	pop 出 _ _，與 `}` 比對		☐ 是　☐ 否

最後 Stack 為空？ ☐ 是　☐ 否　　→　括號配對 ☐ 正確　☐ 錯誤

四、程式碼填空　（共 27 分，每空 ~4 分）

以下是用 Array 實作的 Stack，請填入缺少的部分。

```
class Stack {
private:
    int arr[1000];
    int topIndex;
public:
    Stack() : topIndex( _①_ ) {}          // 空 Stack，-1 代表無元素

    void push(int x) {
        arr[ _②_ ] = x;                   // topIndex 先加 1，再存入
    }

    int pop() {
        if ( _③_ ) throw "Underflow";
        return arr[ _④_ ];                // 回傳頂端值並讓 topIndex -1
    }

    int top() {
        return arr[ _⑤_ ];
    }
```

```
    bool isEmpty() { return topIndex == _⑥_; }
    int  size()    { return topIndex +  _⑦_; }
};
```

空格	填入内容
①	
②	
③	
④	
⑤	
⑥	
⑦	